项目全流程

PART3 RAG

---

0. 一句话定位

本项目实现可扩展 GraphRAG：用 Sentence-Transformers + FAISS 做文本向量检索，并用“图扩展 + 回语料抓证据 chunks”的 graph-guided 检索增强多跳问答；同时支持社区报告机制（community index）与 baseline（vector-only RAG）对比。

0.1 你要在 PPT 强调的核心创新点（3 句话讲清楚）

1. **从“图边塞进 Prompt”升级到“先图后文抓证据 chunks”**：图不是仅做解释，而是参与检索路径。
2. **引入社区报告索引（community index）**：先定位社区再扩展/回语料，减少噪声与检索范围。
3. **引入 PageRank + Louvain**：pagerank 用于候选实体/边排序，community_id 用于同社区语义近邻补全。

---

1. 输入数据（Data）

文本语料

- data/clean_articles.jsonl：文章级（doc_id/title/text），共 50k

图谱结构

- data/vertices.jsonl / data/edges.jsonl
- data/entity_lookup.jsonl：实体名归一（raw/canonical → entity_id） **说明**：entity_id 与 vertices.id 对齐；文本里出现的实体名通过此表归一到图节点

文本抽取三元组（连接文本与图）

- data/extracted_triples.jsonl :包含 doc_id + (subject,predicate,object) ,用于建立对齐与倒排索引 **说明**：这是“回到语料库”的关键，因为它把图实体提及与 doc_id 绑定在一起

图增强产物（重要性 + 社区）

- data/vertices_enriched.jsonl：pagerank + community_id（Louvain）
- data/community_summary.jsonl：社区摘要输入（community report） **说明**：community_summary 不是直接进 LLM，而是先向量化建立 community index

---

2. 关键中间产物（Artifacts）

2.1 文本分块（chunk 级）

- data/clean_article_chunks_50k. jsonl : chunk_id=doc_id#i , 用于 embedding/FAISS **为什么要 chunk** :文章级太长 ;chunk 让检索粒度更准、Prompt 更可控
    - **关键参数**：chunk-chars=1200、overlap-chars=200

2.2 文档→图实体对齐

- data/doc_entities. jsonl : doc_id → entity_ids **如何生成**：

extracted_triples(subject/object) + entity_lookup 映射到 entity_id

2.3 图实体→文档倒排索引（回语料用）

- data/entity_docs.jsonl：entity_id → doc_ids **用途**：graph-guided 扩展实体后，快速定位“哪些文档包含这些实体”

2.4 社区报告向量索引输入

- data/community_chunks.jsonl：把 community_summary 转成可 embedding 的文本块（chunk_id=community:<id>）**用途**：建立 community index（Query → top communities）

---

3. 索引构建（Indexing）

3.1 文本向量索引（FAISS）

- 输入：clean_article_chunks_50k.jsonl
- 输出：indexes/wiki_clean_50k/（index.faiss/docstore.jsonl/idmap.jsonl）
- 技术：Sentence-Transformers embedding + FAISS（GPU 优先）**Embedding 模型**：sentence-transformers/all-MiniLM-L6-v2
    - **相似度**：metric=ip（向量归一化后近似 cosine）
    - **可解释性**：docstore.jsonl 保留 chunk 文本，idmap.jsonl 保留 id 映射

3.2 社区向量索引（community index）

- 输入：community_chunks.jsonl
- 输出：indexes/community_summary/
- 目的：先检索 top 社区，再限制图扩展范围（减少噪声、缩小搜索空间） **检索输出**：top-k community_id

---

4. 在线检索与生成（Retrieval → Prompt → LLM）

4.1 Baseline（Vector-only RAG）

1. Query → FAISS top-k chunks → Vector Context

2. Prompt(Vector Context) → LLM → Answer

- **用途**：对照组，验证“图是否带来多跳提升”

4.2 GraphRAG 版本 1：Hybrid（Vector + Graph edges）

1. Query → FAISS top-k chunks（seed docs）
2. seed docs → doc_entities → 图 1-hop 扩展（Graph Context）
3. Prompt(Vector Context + Graph Context) → LLM → Answer

- **排序**：实体按 pagerank 排序；边按 dst_pagerank 排序
- **同社区扩展（可选）**：从 seed 实体的 community 内补充 top pagerank 实体

4.3 GraphRAG 版本 2（核心）：Graph-guided retrieval（先图后文）

1. Query → FAISS top-k chunks（seed docs）
2. Query → community index → top 社区（可选，但默认启用）
3. seed docs → doc_entities → 选 pagerank 高的实体（pagerank 排序）
4. Louvain 同社区扩展（community 内按 pagerank 补充实体）
5. 图 1-hop/2-hop 扩展（边按 dst_pagerank 排序）
6. expanded entities → entity_docs（回语料候选 doc_id）
7. 候选 doc_id → 抽取 chunks → embedding rerank → Evidence Chunks
8. Prompt(Evidence Chunks + 可选 Graph Context) → LLM → Answer

- **关键点**：Graph 产生“检索路径”，Evidence Chunks 才是最终主要证据
- **为什么要 rerank**：entity→doc 扩展会带噪声，用 query-embedding 对候选 chunks 精排

---

5. 排序与去噪策略（Ranking）

- **pagerank**：实体候选排序、边排序（优先重要节点，增强消歧）
- **community_id**： community index：先定位社区（减少检索范围）
  - intra-community expand：同社区语义近邻补全（提升覆盖与多跳）
- **上限控制**（避免 prompt 爆炸）： community_top_k（默认 3）
  - expand_entities_per_doc（默认 8）

- expand_edges_per_entity（默认 8）
- max_candidate_docs、max_chunks_per_doc（控制回语料候选规模）

---

6. 对比实验（Evaluation framing）

对比维度（建议写入 PPT）：

- 多跳问题覆盖度（是否能补全第二跳信息）
- 幻觉率（是否出现无证据编造）
- 证据可解释性（答案能否指向 evidence chunks / 图关系）

对比组：

- Baseline：cli_rag_hf.py
- GraphRAG Hybrid：cli_graphrag_hf.py
- GraphRAG graph-guided：cli_graphrag_guided_hf.py
- **建议展示**：同一 query 下三种方法的 Prompt 证据块长度、命中 doc/实体数量、以及答案差异

---

7. 工程亮点（Implementation highlights）

- 模块化：索引构建/对齐/倒排索引/社区索引/检索/生成解耦
- 可扩展：支持 50k 文档全量索引，且可增量更新
- 可解释：输出 vector 命中与 graph 扩展边，可追溯证据链
- 可配置：社区 top-k、pagerank 排序、同社区扩展、回语料候选上限均可调参

附录：可直接复制运行的命令块（PPT 素材）

说明：下列命令假设你在项目根目录 /home/mhebd/5003_Group_Project 下运行，并且在 GPU 节点/会话中执行（--device cuda）。

A) 构建文本 chunks（50k 全量）

```
python scripts/make_chunks_from_articles.py \
  --in data/clean_articles.jsonl \
  --out data/clean_article_chunks_50k.jsonl \
  --chunk-chars 1200 \
  --overlap-chars 200 \
  --limit-docs 0
```

B) 构建文本向量索引（FAISS）

```
python -m graphrag_faiss.cli_build_index \
  --chunks data/clean_article_chunks_50k.jsonl \
  --out-dir indexes/wiki_clean_50k \
  --device cuda \
  --metric ip
```

C) 构建 doc_id → entity_ids 对齐表

```
python scripts/build_doc_entity_alignment.py \
  --extracted data/extracted_triples.jsonl \
  --entity-lookup data/entity_lookup.jsonl \
  --out data/doc_entities.jsonl
```

D) 构建 entity_id → doc_ids 倒排索引（graph-guided 回语料）

```
python scripts/build_entity_doc_index.py \
  --extracted data/extracted_triples.jsonl \
  --entity-lookup data/entity_lookup.jsonl \
  --out data/entity_docs.jsonl
```

E) 构建社区索引（community_summary → community index）

```
python scripts/build_community_chunks.py \
  --in data/community_summary.jsonl \
  --out data/community_chunks.jsonl

python -m graphrag_faiss.cli_build_index \
  --chunks data/community_chunks.jsonl \
  --out-dir indexes/community_summary \
  --device cuda \
  --metric ip
```

F) 运行三种 RAG（Baseline vs GraphRAG）

F.1 Baseline：Vector-only RAG

```
python -m graphrag_faiss.cli_rag_hf \
  --index-dir indexes/wiki_clean_50k \
  --q "American actress Love Boat" \
  --llm-model Qwen/Qwen2.5-7B-Instruct \
  --load-in-4bit
```

F.2 GraphRAG（Hybrid）: Vector + Graph edges

```
python -m graphrag_faiss.cli_graphrag_hf \
  --index-dir indexes/wiki_clean_50k \
  --q "American actress Love Boat" \
  --llm-model Qwen/Qwen2.5-7B-Instruct \
  --load-in-4bit
```

F.3 GraphRAG（Graph-guided）: 先图后文（Evidence chunks）

```
python -m graphrag_faiss.cli_graphrag_guided_hf \
  --index-dir indexes/wiki_clean_50k \
  --q "American actress Love Boat" \
  --llm-model Qwen/Qwen2.5-7B-Instruct \
  --load-in-4bit \
  --print-context
```